

Ivan Bondarenko¹, Fedor Tikunov¹, Egor Palkin¹

¹Novosibirsk State University, Novosibirsk, Russia

Correspondence: i.bondarenko@g.nsu.ru

WHAT PROTO-TOKENS ENCODE: DISENTANGLING SEMANTICS AND SYNTAX IN ONE-STEP TEXT RECONSTRUCTION

ABSTRACT. Autoregressive large language models (LLMs) generate text token-by-token, requiring n forward passes to produce a sequence of length n . Recent work, *Exploring the Latent Capacity of LLMs for One-Step Text Reconstruction* (Mezentsev and Osleedets), shows that frozen LLMs can reconstruct hundreds of tokens from only two learned *proto-tokens* in a single forward pass, suggesting a path beyond the autoregressive paradigm. In this paper, we study what information these proto-tokens encode and how they behave under reconstruction and controlled constraints. We perform a series of experiments aimed at disentangling semantic and syntactic content in the two proto-tokens, analyzing stability properties of the e -token, and visualizing attention patterns to the e -token during reconstruction. Finally, we test two regularization schemes for “imposing” semantic structure on the e -token using teacher embeddings, including an anchor-based loss and a relational distillation objective. Our results indicate that the m -token tends to capture semantic information more strongly than the e -token under standard optimization; anchor-based constraints trade off sharply with reconstruction accuracy; and relational distillation can transfer batch-level semantic relations into the proto-token space without sacrificing reconstruction quality, supporting the feasibility of future non-autoregressive seq2seq systems that predict proto-tokens as an intermediate representation.

§1. INTRODUCTION

Autoregressive LLMs generate text in a left-to-right fashion by predicting the next token conditioned on the previously generated context. This approach has a fundamental computational limitation: generating a sequence of length n typically requires n forward passes of the model.

Key words and phrases: proto-tokens and non-autoregressive generation and large language models and one-step text reconstruction and representation distillation.

Non-autoregressive (NAR) text generation aims to reduce this sequential bottleneck by generating multiple tokens (or the whole sequence) in parallel, typically at the cost of additional modeling assumptions, iterative refinement, or distillation from autoregressive teachers. Recent NAR approaches and analyses provide a broad view of the design space and its trade-offs [Li et al.(2024)].

Recent work, *Exploring the Latent Capacity of LLMs for One-Step Text Reconstruction* (Mezentsev and Oseledets), demonstrates that frozen LLMs can reconstruct hundreds of tokens from only two learned proto-tokens in a single forward pass. This observation motivates moving beyond the purely autoregressive paradigm.

We view the frozen-LLM plus learned proto-token interface as a complementary perspective on NAR generation: instead of training a fully new NAR model end-to-end, one can (i) keep a strong pretrained decoder frozen, (ii) predict a small set of continuous proto-tokens, and (iii) decode an entire sequence in one pass. Moreover, our experiments explore explicitly distillation-style constraints (relational distillation) that transfer semantic structure into the proto-token space, connecting this approach to common NAR training recipes [Li et al.(2024)].

Given such a reconstruction mechanism, a natural question arises: can proto-tokens be used not only for reconstruction, but also for *controlled* sequence continuation? To answer this question, we must understand what information the proto-tokens (e and m) contain and how they behave during generation.

Proto-tokens superficially resemble *soft prompts* [Lester et al.(2021)] and *prefix-tuning* [Li and Liang(2021)], but the two settings differ in an essential way. Soft prompts and prefixes are *shared across examples* and are trained on a downstream objective (e.g., classification, generation conditioned on an input); the input text itself is still provided as discrete tokens. In contrast, proto-tokens are *per-example continuous codes* (e_i, m_i) that *replace* the input entirely: the frozen LLM receives only $[e_i, m_i, m_i, \dots, m_i]$ and must reconstruct the target sequence. This makes proto-tokens closer in spirit to a learned per-example compression of text into two vectors than to a conditional prompt, and it is the reason we analyze them through the lens of reconstruction rather than prompt-based task performance.

In this work, we perform the following experimental study of proto-tokens:

- (1) Semantic content encoded in proto-tokens.

- (2) Syntactic content encoded in proto-tokens.
- (3) Stability of the e -token under optimization and constraints.
- (4) Attention visualization to the e -token during reconstruction.
- (5) Experiments on imposing semantic structure on the e -token using a teacher vector.

§2. RELATED WORK

Our work is most directly influenced by [Mezentsev and Oseledets(2025)] (“Exploring the Latent Capacity of LLMs for One-Step Text Reconstruction”). That paper shows that, by feeding two trainable proto-tokens into a frozen LLM and optimizing only their input embeddings, one can reconstruct sequences hundreds of tokens in a single forward pass. Proto-token embeddings are learned by minimizing a cross-entropy reconstruction objective.

For broader context, we refer to recent work on non-autoregressive generation methods and their training strategies, including distillation-based objectives [Li et al.(2024)].

In particular, the input to the frozen LLM can be viewed as a sequence of embeddings where two proto-token embeddings are placed first, followed by a fixed number of repeated m embeddings:

$$X = \left[e, \underbrace{m, m, \dots, m}_{T-1} \right]. \quad (1)$$

Training optimizes only the proto-token embeddings while the model weights remain frozen. Reconstruction is learned with the standard cross-entropy objective:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{T} \sum_{i=1}^T \log p_{\theta}(t_i | X_{<i}) = -\frac{1}{T} \sum_{i=1}^T \log \left(\frac{\exp(z_{i-1,t_i})}{\sum_{c \in V} \exp(z_{i-1,c})} \right). \quad (2)$$

Here $z_{i-1,\cdot}$ are the model logits at position $(i-1)$, V is the vocabulary, and t_i is the i -th target token.

We extend this direction by analyzing what the two proto-tokens encode. In particular, we provide evidence that e and m contain both semantic and syntactic information, though to different degrees. These observations suggest the feasibility of a stable non-autoregressive seq2seq architecture in which a separate model predicts proto-tokens from past context,

and a frozen LLM decodes them into continuations (e.g., for question answering).

§3. METHOD

3.1. Proto-token optimization for reconstruction. We first describe how we obtain proto-token embeddings for one-step reconstruction. Let the target token sequence be $T = [t_1, \dots, t_T]$ and let d be the hidden size of the frozen causal LLM. For each example, we optimize two learnable vectors $e, m \in \mathbb{R}^d$ and build a proto-token-only input embedding sequence

$$X = [e, m, m, \dots, m] \in \mathbb{R}^{T \times d}, \quad (3)$$

where m is repeated $(T - 1)$ times. We feed X to the frozen causal LLM with the standard causal attention mask and obtain logits $Z \in \mathbb{R}^{T \times |V|}$. Following the next-token prediction convention, the logits at position $(i - 1)$ are used to predict token t_i . We optimize only e and m using AdamW [Loshchilov and Hutter(2019)] until the reconstruction token accuracy reaches a threshold (e.g., 0.9) or a maximum number of iterations is reached.

We use two distinct initialization conventions depending on the experimental setting, and we make this distinction explicit because it affects the interpretation of our results.

- **Random initialization** ($e, m \sim \mathcal{N}(0, I)$): used in all experiments *except* the anchor-loss experiment. This includes the baseline reconstruction experiments, the semantic and syntactic augmentation experiments, the attention visualization, the noise robustness study, and the relational distillation experiments.
- **Teacher initialization** ($e_i \leftarrow t_i, m \sim \mathcal{N}(0, I)$): used *only* in the anchor-loss experiment, because initializing e_i far from the teacher vector would make the anchor objective dominate the early optimization dynamics and obscure the trade-off we want to measure.

3.2. Semantic augmentation experiment. To probe semantic content in proto-token representations, for each text in a subset of the dataset we generate 6 lexical and 9 semantic augmentations. For each variant, we optimize proto-token embeddings using the procedure above and then visualize the resulting representations using t-SNE [van der Maaten and Hinton(2008)]. The core hypothesis is: if proto-tokens encode semantic information, then t-SNE projections of proto-tokens for paraphrases of

the same example should cluster together, even when surface form differs [van der Maaten and Hinton(2008)].

Lexical augmentations. Lexical augmentations are based on introducing typos, including: (i) random case flips; (ii) frequent orthographic errors; (iii) common keyboard typing errors; and (iv) random deletion/insertion /repetition/swaps of characters.

Semantic augmentations. Semantic augmentations are paraphrases generated by Qwen3-4B [Qwen Team(2025)].

3.3. Imposing semantic structure on the e -token. Throughout the paper, we use “semantic information” in a deliberately operational sense: we say that a proto-token representation *carries semantic information* if its geometry (pairwise cosine similarity) correlates with the geometry induced by a strong pretrained sentence encoder (Qwen3-Embedding-8B) on the same texts. This is an admittedly narrow proxy — it inherits whatever biases the teacher encoder has and does not directly measure entailment, factual equivalence, or task-level semantic behavior — but it is reproducible, cheap to compute, and, crucially, matches what the anchor and relational objectives actually optimize. Stronger, task-grounded notions of semantics (paraphrase identification, STS, NLI) are left to future work (Section 5.1).

Because the semantic clustering for the e -token was weak in the experiment above, we hypothesize that non-autoregressive optimization admits many good solutions for (e, m) , and random initialization can lead to e vectors that do not preserve semantic relations across examples. This motivates adding additional constraints to encourage semantic structure.

Terminology. Throughout the paper we use the terms *proto-tokens*, *e-token*, and *m-token* consistently. We refer to the decoding setting studied here as *one-step reconstruction* (not “one-step generation”).

We evaluate two regularization schemes:

- (1) **Anchor loss.** To probe whether e can be pushed toward a teacher sentence embedding while retaining reconstruction ability, we add a cosine anchor term and additionally initialize e_i from the teacher vector t_i (obtained from Qwen3-Embedding-8B). Teacher initialization is used *only* in this experiment:

$$\mathcal{L}_{\text{anchor}} = 1 - \cos(e_i, t_i), \quad e_i^{(0)} = t_i. \quad (4)$$

We combine the reconstruction loss with optional regularizers:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{anchor}} \mathcal{L}_{\text{anchor}} + \lambda_{\text{rel}} \mathcal{L}_{\text{rel}}. \quad (5)$$

The anchor term is weighted by a separate hyperparameter in the global loss.

- (2) **Relational distillation.** Because $\mathcal{L}_{\text{rel}}$ is computed over within-batch pairs, its behavior depends on batch composition. We use a simple protocol: each batch contains $B = 6$ examples drawn uniformly at random without replacement from the Dolly training split, independently at each optimization step. We do not enforce paraphrase pairs or topic balancing inside a batch. We additionally compare two settings of m : a *per-example* m_i and a *shared* m used across all examples in the batch; the shared- m setting forces example-specific content to be carried entirely by e_i and is therefore the cleaner test of whether e alone can carry semantic geometry.

We penalize mismatch between pairwise similarity matrices (using MSE/Huber) computed from teacher embeddings t_i and student embeddings e_i within a batch:

$$\mathcal{L}_{\text{rel}} = \frac{1}{B(B-1)} \sum_{i \neq j} \ell \left(S_{ij}^{(E)}, S_{ij}^{(T)} \right), \quad (6)$$

where B is the batch size, $S_{ij}^{(T)} = \cos(t_i, t_j)$ (teacher) and $S_{ij}^{(E)} = \cos(e_i, e_j)$ (student), and $\ell(a, b) = (a - b)^2$ (MSE) or $\ell(a, b) = \text{Huber}(a - b)$.

3.4. Syntactic experiment. To probe syntactic content, we generate seven classes of sentences using an LLM and context-free grammars: (1) simple, (2) complex, (3) simple interrogative, (4) complex interrogative, (5) simple imperative, (6) complex imperative, and (7) one-clause (single-member) sentences. We then test whether t-SNE projections of proto-token representations cluster by syntactic class.

3.5. Attention visualization. We also visualize attention weights to the e -token across layers and heads. Because of causal masking, the e -token does not attend to later tokens; thus we focus on how other positions attend to e during reconstruction.

§4. EXPERIMENTS AND RESULTS

4.1. Attention visualization to the e -token. As described in Section 3.5, we analyze how other positions attend to e across layers and heads.

Mean attention over heads. Figure 1 shows examples of attention to the e -token averaged over heads for different layers.

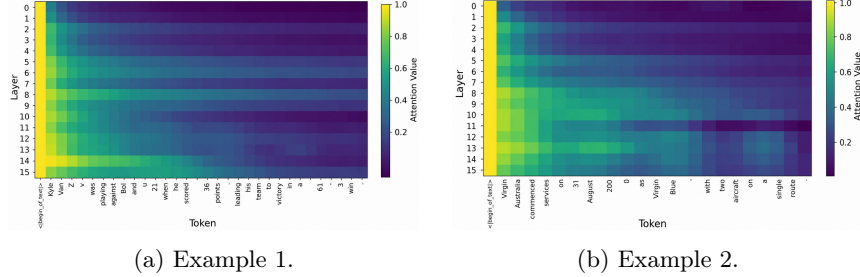


Figure 1. Attention to the e -token averaged over heads (examples).

Layer-wise mean attention heatmaps. We also plot heatmaps of attention to the e -token for selected layers, averaged over heads.

Head-level attention distributions. Finally, we visualize attention to the e -token for individual heads in selected layers (Figure 3).

Overall, we observe that attention to the e -token is typically strongest for early positions and tends to decay for later tokens; the exact pattern varies by layer and head.

4.2. Robustness to additive noise in proto-tokens. We study how additive noise affects reconstruction quality when perturbing the optimized proto-token embeddings, focusing on the stability of the reconstruction process under embedding-level disturbances. Given optimized proto-tokens (e, m) for a text example, we add noise to the e -token and measure reconstruction token accuracy as a function of noise intensity and distribution. Noise distributions. We consider four types of noise:

- (1) Gaussian: $\varepsilon \sim \mathcal{N}(0, \sigma^2)$.
- (2) Uniform: $\varepsilon \sim \mathcal{U}(a, b)$.
- (3) Exponential: $\varepsilon \sim \text{Exp}(\lambda)$.
- (4) Sinusoidal: $\varepsilon = \sin(\omega i + \phi)$ (structured perturbation).

Normalization. We scale the noise to control its magnitude relative to the original embedding norm:

$$\tilde{\varepsilon} = \frac{\varepsilon}{\|\varepsilon\|_2} \cdot \alpha \cdot \|e\|_2, \quad (7)$$

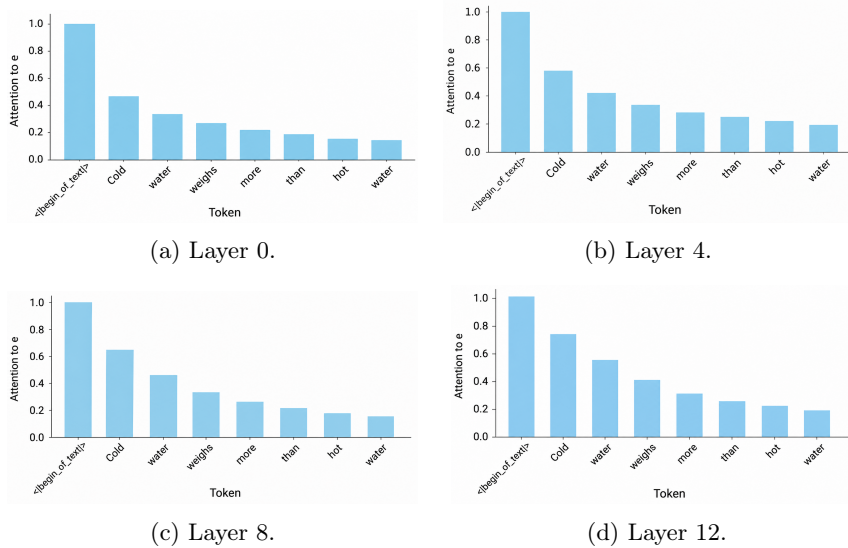


Figure 2. Layer-wise heatmaps of attention to the e -token (averaged over heads).

where $\alpha \in \{0.0, 0.05, 0.1, 0.2, 0.5, 1.0\}$ controls intensity. Figure 4 summarizes how reconstruction token accuracy changes as noise intensity increases for different perturbation types.

As expected, reconstruction quality degrades as the noise level increases; different noise distributions exhibit similar trends in our setup.

4.3. Models and data.

Models and rationale. We use three frozen models, each chosen for a specific role:

- **Llama-3.2-1B** as the frozen decoder for proto-token optimization. We choose a 1B-parameter decoder because per-example proto-token optimization is expensive (hundreds to thousands of optimizer steps *per text*), and a 1B model allows us to run the full grid of semantic, syntactic, robustness, and distillation experiments within our compute budget while remaining a modern pretrained causal LM.

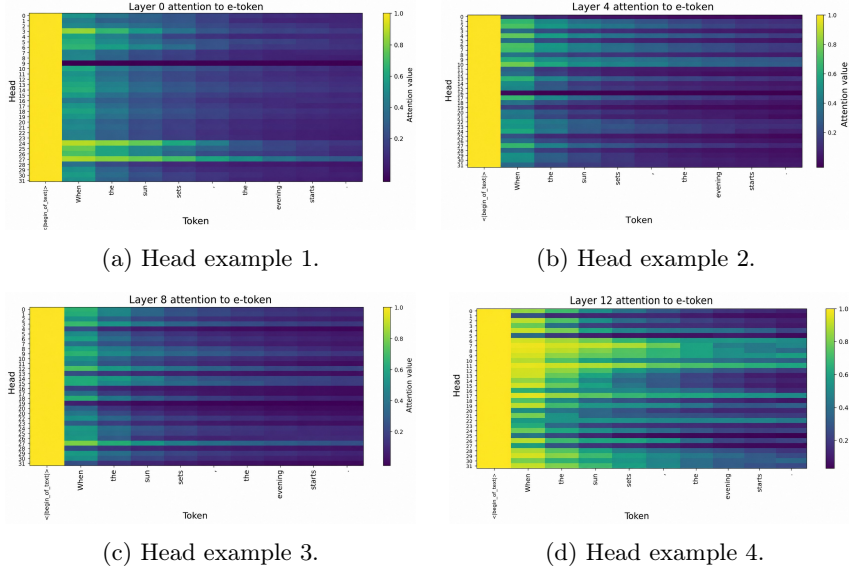


Figure 3. Head-level attention to the e -token for selected layers/heads (examples).

- **Qwen3-4B** as the paraphrase and synthetic-sentence generator. Paraphrase quality and instruction-following matter here far more than decoder-level reconstruction, which motivates a larger, instruction-tuned model from a different family than the decoder (to reduce shared biases between the decoder we probe and the data-generation model).
- **Qwen3-Embedding-8B** as the teacher for the anchor and relational objectives. We pick a strong, publicly available sentence encoder so that the teacher geometry is a meaningful target; using an encoder from the same family as the generator ensures the paraphrase-generation and semantic-target pipelines are internally consistent.

Generalization of our findings to other decoder families and scales is left to future work.

Data. For the semantic experiment, we use the databricks-dolly-15k dataset of instruction-following pairs [Dolly(2023)]. We generate paraphrases via

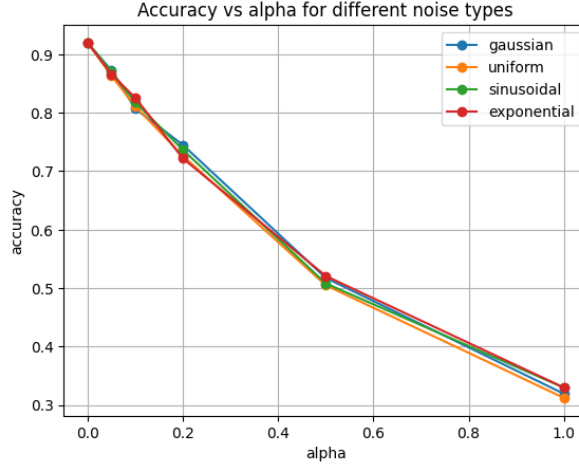


Figure 4. Reconstruction token accuracy as a function of noise intensity α for different noise types. Each coloured curve corresponds to one noise distribution (Gaussian, Uniform, Exponential, Sinusoidal); the solid blue curve shows the mean over distributions, and shaded regions (if present) indicate the range across examples.

Qwen3-4B and typo-based variants via the `augmentex` library [Augmentex(2023)]. For semantic-imposition and attention-visualization experiments, we also use Dolly. For the syntactic experiment, we use fully synthetic data generated by Qwen3-4B and context-free grammars; the grammar vocabulary is built from the 500 most frequent words in the Brown corpus categories “news”, “editorial”, “learned”, and “reviews” [Francis and Kučera(1979)], with POS tags assigned via WordNet matches [Miller(1995)].

Optimization details. Proto-token embeddings are optimized with AdamW [Loshchilov and Hutter(2019)] using learning rate 0.01, $\beta_1 = \beta_2 = 0.9$, and weight decay 0.01. We optimize each example until token accuracy reaches 0.9 or for at most 2000 iterations. Initialization follows the conventions described in Section 3.1.

4.4. Semantic content: e vs. m . From t-SNE visualizations, we observe that the m -token embeddings (Figure 6) exhibit stronger semantic clustering than e (Figure 5) under standard optimization. This suggests that m carries more semantic information about the sentence content, while e may encode other factors (or be less identifiable across solutions).

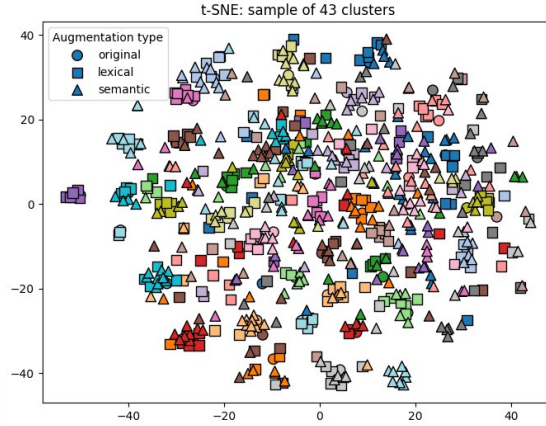


Figure 5. t-SNE visualization of optimized e -token embeddings for original, lexical, and semantic augmentations.

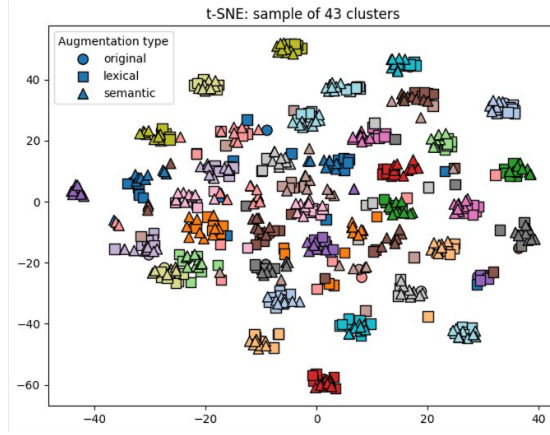


Figure 6. t-SNE visualization of optimized m -token embeddings for original, lexical, and semantic augmentations.

4.5. Imposing semantics on e .

Anchor loss. With a small anchor weight (e.g., 0.02 multiplying the anchor term in the global loss), Figure 7 shows that with a small anchor weight reconstruction remains mostly successful across target lengths. However, the final cosine similarity between e and the teacher vector remains close to zero in our runs, as shown in Figure 8. Increasing the anchor weight (e.g., to 0.5) substantially degrades reconstruction accuracy (Figure 10) while increasing the final similarity (Figure 9). Overall, these results do not support the hypothesis that e can be forced to match an off-the-shelf sentence embedding while preserving reconstruction quality.

Relational distillation. Using relational distillation, we find that even without constraints there is weak semantic structure in e -tokens, but it becomes more apparent when using a shared m -token (Figure 11). Importantly, adding the relational constraint preserves the ability to reconstruct text accurately (Figure 12) while significantly improving correlation between teacher and student similarity matrices across batches (Figure 13) (batch size 6). This indicates that relational distillation can transfer semantic relations into the proto-token space without sacrificing reconstruction.

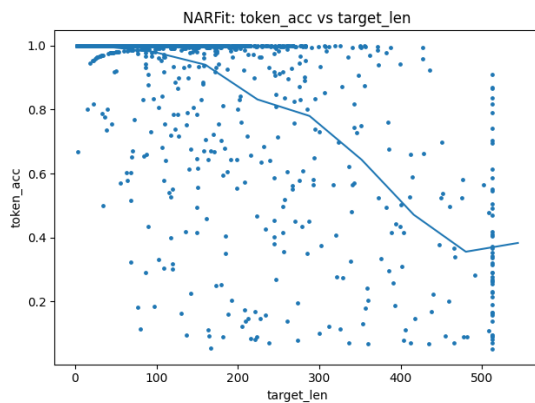


Figure 7. Reconstruction token accuracy vs. target length under anchor regularization with a small weight.

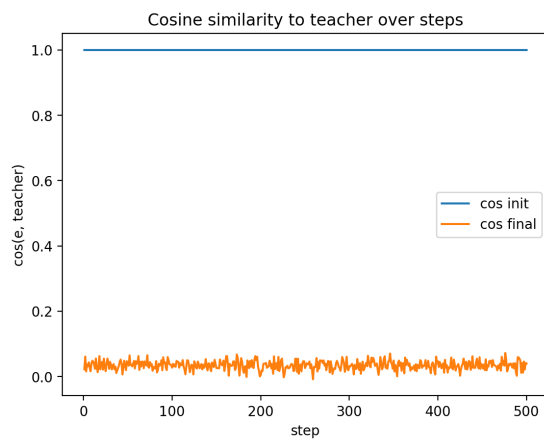


Figure 8. Cosine similarity between e and the teacher embedding over optimization steps (anchor).

4.6. Syntactic content and attention patterns. Our syntactic clustering and attention analyses suggest that proto-tokens also encode information correlated with syntactic structure. These observations support

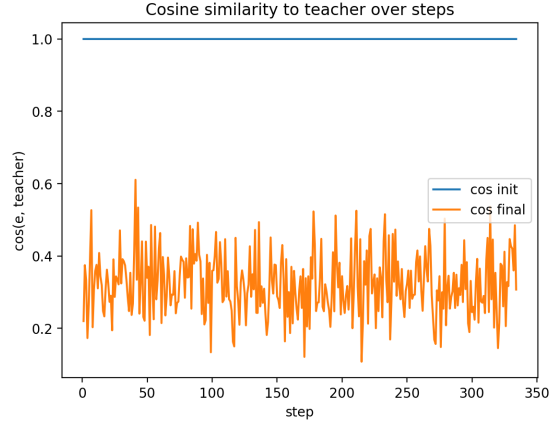


Figure 9. Cosine similarity between e and the teacher embedding over optimization steps for a larger anchor weight.

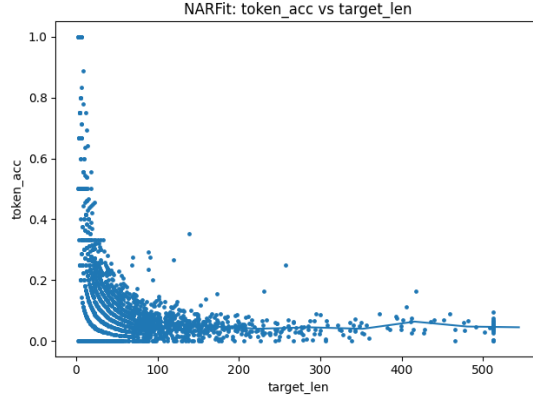


Figure 10. Token accuracy vs. target length for a larger anchor weight, showing degraded reconstruction.

the working hypothesis that, if m is more semantically aligned, then e may

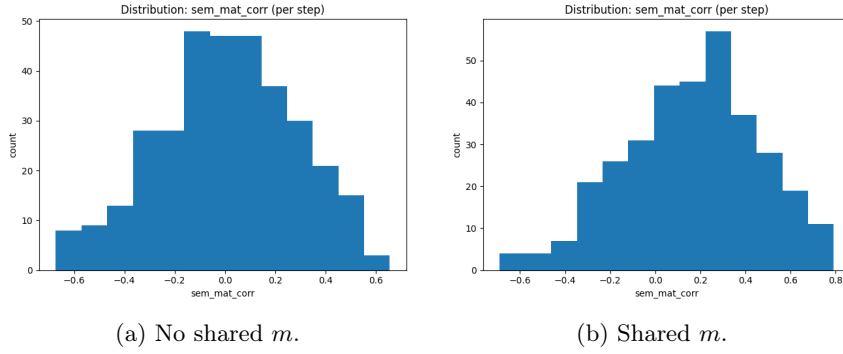


Figure 11. Distributions of semantic-matrix correlation per step for relational distillation, with and without a shared m -token.

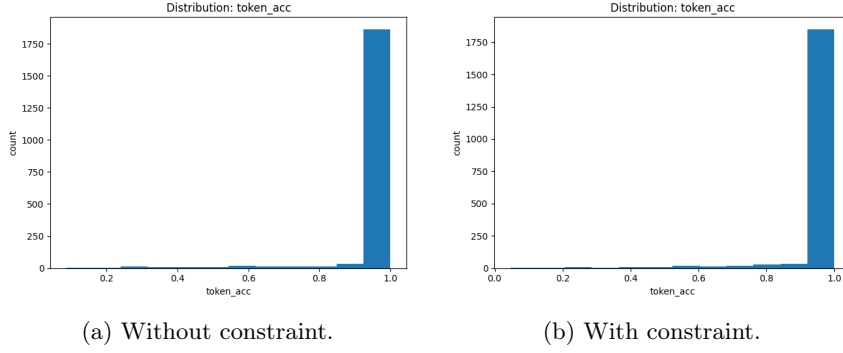


Figure 12. Token-accuracy distributions with and without the relational distillation constraint.

carry more syntactic information; we leave a detailed quantitative study as future work.

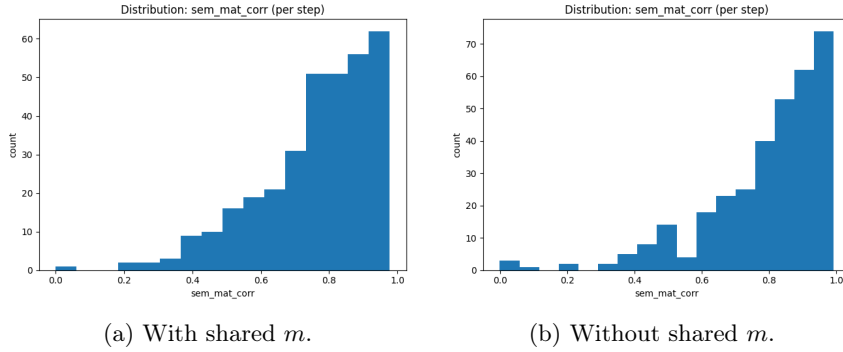


Figure 13. Improved semantic-matrix correlation distributions after applying the relational distillation objective.

§5. DISCUSSION AND CONCLUSION

We studied proto-tokens used for one-step reconstruction with frozen LLMs and investigated what information they encode. Our experiments indicate that:

- Under unconstrained optimization, the m -token tends to capture semantic information more strongly than the e -token.
- Anchor-based regularization that forces e toward a teacher sentence embedding introduces a sharp trade-off: stronger anchoring improves similarity but quickly harms reconstruction accuracy, and weak anchoring does not reliably align e with the teacher space.
- Relational distillation is a more promising alternative: it transfers batch-level semantic relations into the proto-token space while preserving high reconstruction performance.
- Proto-tokens appear to encode syntactic information as well, and attention patterns suggest that the model actively uses the e -token during decoding.

These findings support the feasibility of a non-autoregressive seq2seq pipeline in which a lightweight predictor maps context to proto-tokens, and a frozen LLM decodes them into text.

5.1. Future Work. The picture that emerges from our experiments is only a first sketch, and it invites several natural continuations.

The most immediate question is how to move from representational geometry to operational meaning. Similarity in a teacher embedding space is a convenient proxy, but it is not the same as semantics in any task-grounded sense. Probing classifiers, causal interventions that swap e or m between paraphrases, and evaluation on paraphrase, entailment, and retrieval tasks would together turn our current correlational evidence into a sharper claim about what proto-tokens actually encode.

A second, deeper question concerns identifiability. The optimization that produces (e, m) admits many good solutions, and the specialization we observe — with m carrying most of the semantic load — may be as much a property of the landscape and its starting point as of the representation itself. Understanding how initialization, decoder family and scale, and the number of proto-tokens shape this specialization would clarify what part of our story is intrinsic to the one-step reconstruction idea and what part is an artifact of a particular training recipe.

Finally, the proto-token interface is interesting mainly to the extent that it can be *predicted* from context rather than optimized per example. Training an upstream model that maps context to (e, m) , whether by distillation from optimized proto-tokens or by joint training with the frozen decoder, is the step that would turn the present analysis into an actual non-autoregressive generation system. We see the current paper as clearing the ground for that step: once it is known what proto-tokens can and cannot encode, and under what constraints, designing a predictor for them becomes a much better-posed problem.

APPENDIX §A. TRAINING AND OPTIMIZATION DETAILS

A.1. Prototype Token Initialization. As described in Section 3.1, we use two initialization conventions. In all experiments except the anchor-loss study, both e and m are initialized from a standard normal distribution $\mathcal{N}(0, I)$. In the anchor-loss experiment, e_i is instead initialized from the teacher vector t_i , while m remains sampled from $\mathcal{N}(0, I)$.

A.2. Optimization. Embeddings were optimized using the AdamW optimizer with the following hyperparameters:

- Learning rate: 0.01
- $\beta_1 = 0.9$
- $\beta_2 = 0.9$
- Weight decay: 0.01

All other parameters were kept at default values.

APPENDIX §B. SEMANTIC AUGMENTATION EXPERIMENT

B.1. Lexical augmentations. Character-level lexical augmentation was performed using the `augmentex` library with the following configuration:

- Character modification probability: 0.3
- Minimum number of augmentations per sentence: 1
- Maximum number of augmentations per sentence: 5
- Maximum character repetition factor: 3
- Keyboard layout for typo simulation: PC

B.2. Semantic augmentations. Semantic paraphrases were generated using the Qwen3-4B language model. The decoding parameters were:

- Temperature: 0.7
- Top-p: 0.9

Prompt template:

You are a helpful assistant that generates high-quality paraphrases.
Paraphrases must preserve the original meaning and factual content.
Do not add new information.
Use different wording and sentence structures.

Original:

The model was trained on a large dataset.

Paraphrases:

1. The model was trained using a large amount of data.
2. A large dataset was used to train the model.
3. The model learned from a very large collection of data.

Original:

Virgin Australia commenced services on 31 August 2000 as Virgin Blue, with two aircraft on a single route.

Paraphrases:

1. Virgin Australia began operations on 31 August 2000 under the name Virgin Blue, operating two aircraft on a single route.
2. The airline launched on 31 August 2000 as Virgin Blue,

starting with just two planes and one route.
 3. Operations began on 31 August 2000 when the company, then known as Virgin Blue, entered service with two aircraft on a single route.

Original:

The system returns an error when the input format is incorrect.

Paraphrases:

1. The system produces an error if the input format is invalid.
2. An incorrect input format causes the system to return an error.
3. The system fails with an error when the input format is wrong.

Original:

{paraphrase}

Return exactly {paraphrase_number} paraphrases as a JSON array of strings.
 Only return valid JSON, do not include numbers or any additional text.

APPENDIX §C. SYNTACTIC AUGMENTATION EXPERIMENT

To probe syntactic content, we generated seven sentence classes using an LLM and context-free grammars.

C.1. Lexical Resources for CFG Generation. Lexical resources for CFG generation were derived as follows:

- (1) The 500 most frequent words were extracted from the Brown corpus, restricted to the categories **news**, **editorial**, **learned**, and **reviews**.
- (2) Each word was assigned a part of speech using WordNet synsets and POS tagging to ensure grammatical correctness.

C.2. Context-Free Grammars. Below we list the production rules for each of the seven sentence classes. In all grammars, **N**, **V**, **Adj**, and **Adv** expand to open-class terminals sampled from the lexicon described above;

Table 1. Closed-class terminal symbols used in all grammars.

Category	Terminals
Det	<i>the, a, an, this, that, these, those, every, each, some, any, my, your, his, her, its, our, their</i>
Prep	<i>in, on, with, for, by, at, over, under, between, among, before, after, during, since, until</i>
Conj	<i>and, but, because, while, or, so, yet, nor, although, though, since, unless, if, even though, as soon as</i>
Aux	<i>do, does, did, will, would, can, could, should, may, might, must, have, has, had, is, are, was, were, be, being, been</i>

Det, Prep, Conj, and Aux expand to the closed-class terminals listed in Table 1.

1. Simple declarative.

$$\begin{aligned} S &\rightarrow NP \ VP \\ NP &\rightarrow Det \ N \\ VP &\rightarrow V \end{aligned}$$

2. Complex declarative.

$$\begin{aligned} S &\rightarrow NP \ ' ; ' \ PartP \ ' ; ' \ VP \\ NP &\rightarrow Det \ N \mid N \ PP \\ PartP &\rightarrow Adj \ PP \\ PP &\rightarrow Prep \ NP \\ VP &\rightarrow V \ AdvP \\ AdvP &\rightarrow Adv \ Conj \ Adv \end{aligned}$$

3. Simple interrogative.

$$\begin{aligned} S &\rightarrow Aux \ NP \ VP \ ' ? ' \\ NP &\rightarrow Det \ N \mid N \\ VP &\rightarrow V \end{aligned}$$

4. Complex interrogative.

$$\begin{aligned} S &\rightarrow \text{Aux NP VP } \text{'?'} \\ NP &\rightarrow \text{Det Adj N} \mid \text{Det N} \\ VP &\rightarrow \text{V Adv} \mid \text{V PP} \\ PP &\rightarrow \text{Prep NP} \end{aligned}$$

5. Simple imperative.

$$\begin{aligned} S &\rightarrow \text{VP} \\ VP &\rightarrow \text{V} \mid \text{V NP} \mid \text{V Adv} \\ NP &\rightarrow \text{Det N} \mid \text{N} \end{aligned}$$

6. Complex imperative.

$$\begin{aligned} S &\rightarrow \text{VP} \\ VP &\rightarrow \text{V NP} \mid \text{V AdvP} \mid \text{V NP PP} \mid \text{V AdvP PP} \\ \text{AdvP} &\rightarrow \text{Adv} \mid \text{Adv Conj Adv} \\ NP &\rightarrow \text{Det N} \mid \text{N} \mid \text{Adj NP} \mid \text{N PP} \\ PP &\rightarrow \text{Prep NP} \end{aligned}$$

7. One-clause (single-member).

$$\begin{aligned} S &\rightarrow \text{NP} \mid \text{VP} \\ NP &\rightarrow \text{Det N} \mid \text{N} \mid \text{Adj NP} \mid \text{N PP} \\ VP &\rightarrow \text{V} \mid \text{V NP} \mid \text{V Adv} \mid \text{V AdvP} \mid \text{V NP PP} \\ \text{AdvP} &\rightarrow \text{Adv} \mid \text{Adv Conj Adv} \\ PP &\rightarrow \text{Prep NP} \end{aligned}$$

C.3. LLM-Based Sentence Generation. For each of the seven syntactic classes, sentences were also generated using Qwen3-4B to complement the CFG-generated data with more naturalistic examples. The model was queried with class-specific prompts that specified the target syntactic structure, an illustrative example, and surface-level constraints (word count, punctuation). Each prompt requested a fixed number of sentences returned as a JSON array.

Decoding parameters.

- Temperature: 0.7
- Top- p : 0.9

Prompt template. All prompts share the following structure (shown here for the *simple declarative* class; class-specific fields are summarized in Table 2):

```
Generate {sample_size} English sentences.

Sentence type:
- Simple declarative sentence
- One subject and one predicate
- No subordinate clauses

Structure example:
"The child sleeps."

Requirements:
- Focus on syntax, not meaning
- Use simple common words
- 3-6 words
- Return exactly {sample_size} paraphrases as a JSON array
  of strings.
Only return valid JSON, do not include numbers or any additional
text.
Return JSON only. Start with '[' and end with ']'.
```

Class-specific prompt fields. Table 2 lists the fields that vary across syntactic classes; all other parts of the prompt remain identical.

Table 2. Class-specific fields in the LLM generation prompts.

Class	Type description	Structure example	Length
Simple declarative	One subject and one predicate; no subordinate clauses	“The child sleeps.”	3–6 words
Complex declarative	One main clause; one modifier phrase separated by commas; no conjunctions between clauses	“The child, tired after the day, sleeps.”	6–12 words
Simple interrogative	Yes/no question; no modifiers; auxiliary verb (<i>do/does</i>)	“Do birds sing?”	3–6 words
Complex interrogative	Yes/no question; one modifier phrase; no subordinate clauses	“Does the tired child sleep at night?”	6–12 words
Simple imperative	No subject; no modifiers; base verb form	“Run.” / “Close the door.”	1–4 words
Complex imperative	No subject; one modifier phrase; no conjunctions; base verb form	“Open the door in silence.”	4–10 words
One-clause (single-member)	Only a noun phrase <i>or</i> a verb phrase; no subject–predicate structure	“Night.” / “Running.” / “Silence.”	1–3 words

REFERENCES

- Augmentex(2023). Augmentex contributors. *augmentex*: Text augmentation library. Software release, 2023.
- Francis and Kučera(1979). W. Nelson Francis and Henry Kučera. *Brown Corpus Manual*. Department of Linguistics, Brown University, 1979.
- Dolly(2023). Databricks. Databricks Dolly 15k dataset. Dataset release, 2023.
- Li et al.(2024). Yafu Li, Huajian Zhang, Jianhao Yan, Yongjing Yin, and Yue Zhang. What have we achieved on non-autoregressive translation? *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. <https://aclanthology.org/2024.findings-acl.452/>.
- Loshchilov and Hutter(2019). Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Mezentsev and Oseledets(2025). Gleb Mezentsev and Ivan Oseledets. Exploring the latent capacity of LLMs for one-step text generation. arXiv preprint, 2025. arXiv:2505.21189. <https://arxiv.org/abs/2505.21189>.

- Miller(1995). George A. Miller. WordNet: A lexical database for English. *Communications of the ACM*, 38(11):39–41, 1995.
- van der Maaten and Hinton(2008). Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9(Nov):2579–2605, 2008.
- Qwen Team(2025). Qwen Team Qwen3 Technical Report. arXiv preprint, 2025. arXiv:2505.09388. <https://arxiv.org/abs/2505.09388>.
- Lester et al.(2021). Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- Li and Liang(2021). Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, 2021.

Иван Бондаренко, Фёдор Тикунов, Егор Палкин

Что кодируют прототокены: Разделение семантики и синтаксиса в одношаговой реконструкции текста

Аннотация. В работе исследуется, какую семантическую и синтаксическую информацию несут прото-токены, используемые для одношаговой реконструкции текста с помощью замороженной большой языковой модели. Мы анализируем содержимое прото-токенов e и m , изучаем устойчивость e -токена к ограничениям и шуму, визуализируем паттерны внимания к e -токену и рассматриваем два способа навязывания семантической структуры: якорную регуляризацию и реляционную дистилляцию. Эксперименты показывают, что при стандартной оптимизации m -токен обычно сильнее отражает семантическое содержание, чем e -токен; жёсткая якорная привязка ухудшает качество реконструкции; а реляционная дистилляция позволяет переносить батчевые семантические отношения в пространство прото-токенов без заметной потери качества восстановления текста. Полученные результаты поддерживают идею неавторегрессивных seq2seq-систем, в которых прото-токены служат промежуточным непрерывным представлением.

Ключевые слова: прото-токены; неавторегрессивная генерация; большие языковые модели; одношаговая реконструкция текста; дистилляция представлений.